



Water Level Detection and Flood Alert System

EXERCISE NO – 05

HARRISH T M	– 24MER016
KAPIL P	– 24MER022
RAMPRASATH D	– 24MER047
SARAVANAKARTHIK S R	– 24MER055
SETHUPATHI M	– 24MER056

5. Water Level Detection and Flood Alert System

Project Overview:

Flooding is one of the most common natural disasters and can cause serious damage to houses, agricultural land, roads, industries, electrical systems, and human life. In many situations, flooding occurs because the water level rises beyond a safe limit and people are not warned early enough.

The Water Level Detection and Flood Alert System is an IoT-based monitoring and warning system designed to continuously measure the level of water and provide an immediate alert when the water level becomes dangerous.

The proposed system uses an ESP8266 NodeMCU as the main controller. An HC-SR04 ultrasonic sensor is used to measure the distance between the sensor and the surface of the water. From this distance, the system calculates the actual water level.

An OLED display provides real-time information about the measured water level and the current condition. Two LEDs provide visual indications, while a buzzer provides an audible warning when the water reaches a dangerous level.

A GSM module is included to provide remote notification. When the water level reaches the critical flood threshold, the system can send an SMS alert to a predefined mobile number.

Main Objective:

The main objective of the project is to develop a low-cost automatic system that can:

- Continuously monitor water level.
- Detect abnormal or rapidly rising water levels.
- Display the water-level information on an OLED.
- Provide visual warning using LEDs.
- Produce an audible alarm using a buzzer.
- Send an SMS alert through a GSM module during critical conditions.
- Reduce the time required for people to respond to rising water levels.
- Provide an easily deployable solution for tanks, drainage systems, low-lying areas, and flood-prone locations.

Problem Statement:

Flooding can occur due to heavy rainfall, overflowing rivers, blocked drainage systems, dam overflow, or sudden increases in water flow.

Traditional water-level monitoring methods often depend on manual observation. A person may have to physically inspect the water level and determine whether it has reached a dangerous point. This method has several disadvantages.

Problems with conventional monitoring

- Manual monitoring is time-consuming.
- Continuous observation is difficult.
- Warnings may be delayed.
- People may not be present at the monitoring location.
- Night-time monitoring is difficult.
- Remote locations are difficult to monitor.
- There may be no automatic warning mechanism.
- Sudden water-level increases may go unnoticed.

Therefore, there is a need for an automatic system capable of continuously monitoring water level and immediately notifying people when a critical level is reached.

Problem Definition

The proposed project addresses this problem by using an ultrasonic sensor to measure the water level automatically. The ESP8266 processes the sensor data and determines whether the water condition is normal, warning, or critical.

When a critical level is detected, the system activates the buzzer and warning LED and sends an SMS notification through the GSM module.

Proposed Solution:

The proposed solution is an automatic Water Level Detection and Flood Alert System based on the ESP8266 NodeMCU.

The HC-SR04 ultrasonic sensor is installed above the water surface. The sensor sends an ultrasonic pulse toward the water. The pulse is reflected from the water surface and returns to the sensor.

The ESP8266 calculates the distance using the echo time.

The water level is calculated using:

Water Level = Reference Distance – Measured Distance

For example, assume the distance from the ultrasonic sensor to the bottom/reference point is 100 cm.

If the measured distance from the sensor to the water surface is 70 cm:

Water Level = $100 - 70 = 30$ cm

If the measured distance decreases to 20 cm:

Water Level = $100 - 20 = 80$ cm

This means that the water level has increased.

System Architecture:

The HC-SR04 sensor measures the distance to the water surface. The measured data is transferred to the ESP8266.

The ESP8266 calculates the water level and compares it with predefined threshold values.

Depending on the result:

Normal Condition

The green LED remains ON and the OLED displays the normal water-level condition.

Warning Condition

The warning indication is activated. The buzzer can operate intermittently and the OLED displays a warning message.

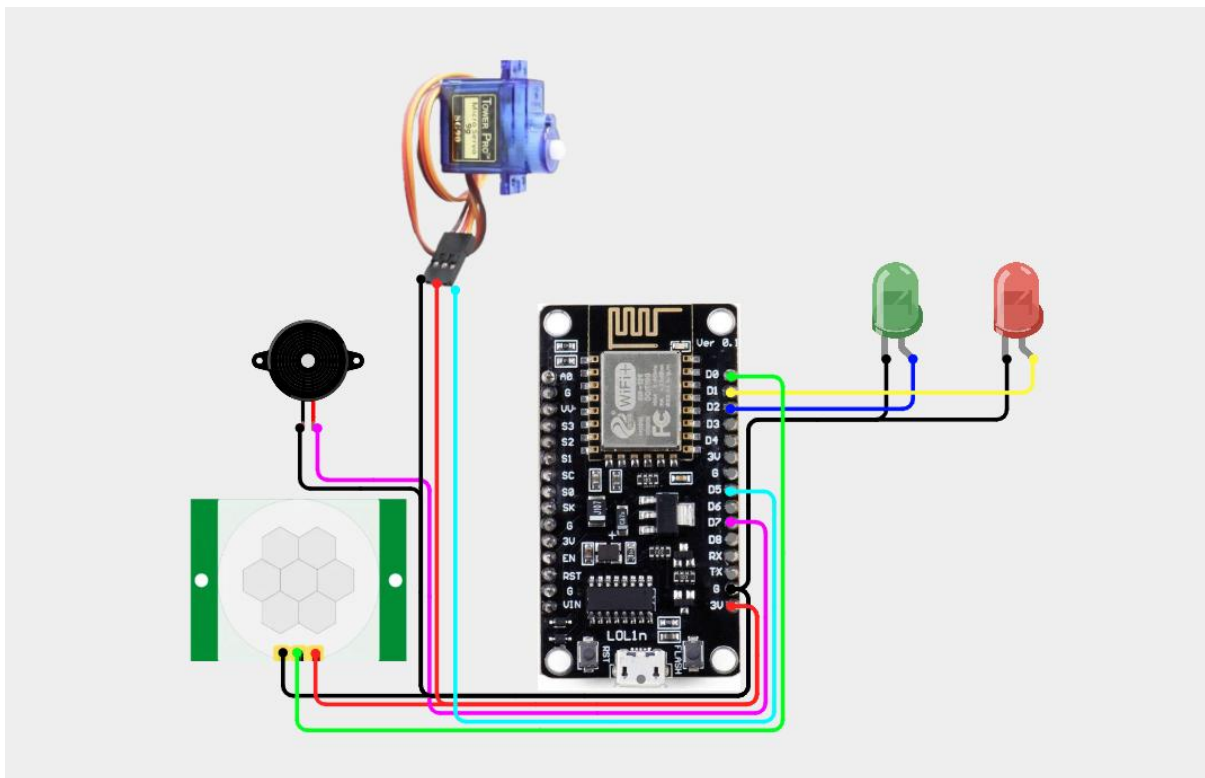
Critical Condition

The red LED and buzzer are activated. The GSM module sends an SMS to the predefined phone number informing the user that the water level has reached the flood/critical threshold.

Circuit Table:

Component	Pin	ESP8266 Pin	Purpose
HC-SR04	VCC	VIN/5V supply	Sensor power
HC-SR04	GND	GND	Ground
HC-SR04	TRIG	D7 / GPIO13	Trigger pulse
HC-SR04	ECHO	D8 / GPIO15	Echo measurement
OLED	VCC	3.3V	OLED power
OLED	GND	GND	Ground
OLED	SCL	D1 / GPIO5	I ² C clock
Green LED	Anode	D3 / GPIO0	Normal indication
Green LED	Cathode	GND	Ground
Red LED	Anode	D4 / GPIO2	Critical indication
Red LED	Cathode	GND	Ground
GSM	TX	D5 / GPIO14	GSM data to ESP8266

Circuit Design:



Algorithm:

- Start the system.
- Initialize ESP8266 GPIO pins.
- Initialize the OLED display.
- Initialize the GSM communication.
- Set the water-level threshold values.
- Trigger the HC-SR04 sensor.
- Measure the echo duration.
- Calculate the distance to the water surface.
- Calculate the water level.
- Display the water level on the OLED.
- Compare the water level with the predefined thresholds.
- If the level is normal:
 - Turn ON green LED.
 - Turn OFF red LED.
 - Turn OFF buzzer.
- If the level is in warning range:
 - Activate warning indication.
 - Operate buzzer intermittently.
 - Display WARNING.
- If the level reaches the critical range:
 - Turn ON red LED.
 - Activate buzzer.
 - Display FLOOD/CRITICAL.
 - Send SMS through GSM.
- Repeat the measurement continuously.

Coding:

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <SoftwareSerial.h>

#define WIFI_SSID "K-APIL"
#define WIFI_PASS "kapil.2333"

#define IO_USERNAME "saravanakarthiksr"
#define IO_KEY      "aio_idPg82y91fmNVXL0iJA71zKwH97q"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);

AdafruitIO_Feed *waterDistance = io.feed("water-distance");
AdafruitIO_Feed *waterLevel    = io.feed("water-level");
AdafruitIO_Feed *floodStatus   = io.feed("flood-status");
AdafruitIO_Feed *gsmStatus     = io.feed("gsm-status");

#define TRIG_PIN  14  // D5
#define ECHO_PIN  12  // D6

#define GREEN_LED  5   // D1
#define RED_LED    4   // D2
#define BUZZER     0   // D3

#define GSM_RX     13  // D7 <- GSM TX
#define GSM_TX     15  // D8 -> GSM RX

SoftwareSerial gsm(GSM_RX, GSM_TX);

const char PHONE_NUMBER[] = "+91XXXXXXXXXXXX";

const float TANK_HEIGHT = 30.0;

const float WARNING_PERCENT = 70.0;
const float FLOOD_PERCENT  = 85.0;
```

```
// Prevent repeated SMS during the same flood
bool smsAlreadySent = false;

SETUP void setup()
{
  Serial.begin(115200);
  gsm.begin(9600);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  // Initial condition
  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, LOW);

  Serial.println();
  Serial.println("");
  Serial.println(" WATER LEVEL FLOOD ALERT SYSTEM");
  Serial.println("");

  Serial.println("Initializing GSM...");

  gsm.println("AT");
  delay(1000);

  gsm.println("AT+CMGF=1");
  delay(1000);

  Serial.println("GSM Ready");

  Serial.println("Connecting to Adafruit IO...");

  io.connect();

  while (io.status() < AIO_CONNECTED)
  {
    Serial.print(".");
```



```

    delay(500);
}

Serial.println();
Serial.println("Adafruit IO Connected!");

// Initial GSM status
gsmStatus->save("NOT SENT");

Serial.println("");
}

float getDistance()
{
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);

    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);

    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000);

    if (duration == 0)
    {
        return -1;
    }

    float distance = duration * 0.0343 / 2.0;

    return distance;
}

float calculateWaterLevel(float distance)
{
    float level = TANK_HEIGHT - distance;

    if (level < 0)
        level = 0;

    if (level > TANK_HEIGHT)
        level = TANK_HEIGHT;
}

```

```
    return level;
}
float calculatePercentage(float level)
{
    float percentage = (level / TANK_HEIGHT) * 100.0;

    if (percentage < 0)
        percentage = 0;

    if (percentage > 100)
        percentage = 100;

    return percentage;
}
```

```
bool sendFloodSMS(float distance, float percentage)
{
    Serial.println();
    Serial.println("");
    Serial.println("FLOOD DETECTED");
    Serial.println("SENDING GSM SMS...");
    Serial.println("");

    gsm.println("AT+CMGF=1");
    delay(1000);

    gsm.print("AT+CMGS=\"");
    gsm.print(PHONE_NUMBER);
    gsm.println("\");

    delay(1500);

    // SMS content
    gsm.print("FLOOD ALERT! ");
    gsm.print("Water Level: ");
    gsm.print(percentage, 1);
    gsm.print("%, Distance: ");
    gsm.print(distance, 1);
    gsm.print(" cm.");

    delay(500);

    // CTRL + Z
    gsm.write(26);
}
```

```

// Wait for GSM response
unsigned long startTime = millis();
String response = "";

while (millis() - startTime < 10000)
{
    while (gsm.available())
    {
        char c = gsm.read();
        response += c;
        Serial.write(c);
    }

    delay(10);
}

Serial.println();

// Check GSM response
if (response.indexOf("+CMGS:") >= 0)
{
    Serial.println("GSM STATUS: SMS SENT");

    gsmStatus->save("SENT");

    return true;
}
else
{
    Serial.println("GSM STATUS: SMS NOT SENT");

    gsmStatus->save("NOT SENT");

    return false;
}
}

String updateAlert(float percentage)
{
    String status;

    if (percentage < WARNING_PERCENT)
    {
        digitalWrite(GREEN_LED, HIGH);
        digitalWrite(RED_LED, LOW);
        digitalWrite(BUZZER, LOW);
    }
}

```

```

    status = "NORMAL";
}

else if (percentage < FLOOD_PERCENT)
{

    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    digitalWrite(BUZZER, LOW);

    status = "WARNING";
}

else
{
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    digitalWrite(BUZZER, HIGH);

    status = "FLOOD";
}

return status;
}

void loop()
{
    io.run();

    float distance = getDistance();

    if (distance >= 0)
    {
        float level = calculateWaterLevel(distance);

        float percentage = calculatePercentage(level);

        String status = updateAlert(percentage);

        Serial.println();
        Serial.println("");

        Serial.print("Distance    : ");
        Serial.print(distance, 1);

```

```
Serial.println(" cm");
```

```
Serial.print("Water Level : ");  
Serial.print(level, 1);  
Serial.println(" cm");
```

```
Serial.print("Water Level : ");  
Serial.print(percentage, 1);  
Serial.println(" %");
```

```
Serial.print("Flood Status : ");  
Serial.println(status);
```

```
    waterDistance->save(distance);  
    waterLevel->save(percentage);  
    floodStatus->save(status);
```

```
if (status == "FLOOD")  
{  
    // Send SMS ONLY during flood  
    if (smsAlreadySent == true)  
    {  
        bool result = sendFloodSMS(distance, percentage);  
  
        if (result)  
        {  
            Serial.println("FINAL GSM STATUS: SENT");  
        }  
        else  
        {  
            Serial.println("FINAL GSM STATUS: NOT SENT");  
        }  
  
        smsAlreadySent = true;  
    }  
    else  
    {  
        Serial.println("FLOOD SMS ALREADY SENT");  
        Serial.println("No additional SMS sent.");  
  
        gsmStatus->save("SENT");  
    }  
}  
else
```

```
{
  // NORMAL or WARNING
  Serial.println("GSM STATUS: NOT SENT");

  gsmStatus->save("NOT SENT");

  // Reset SMS permission only when water is NORMAL
  if (status == "NORMAL")
  {
    smsAlreadySent = false;
  }
}

Serial.println("");
}
else
{
  Serial.println("Ultrasonic Sensor Error!");
}

delay(5000);
}
```

System Working:

The system works continuously after power is supplied.

Step 1: System Initialization

When the system is powered ON, the ESP8266 initializes all connected components.

The OLED displays:

Water Level

Flood Alert System

System Starting...

Step 2: Water-Level Measurement

The HC-SR04 sends an ultrasonic pulse toward the water surface.

The signal is reflected by the water and returns to the sensor.

The ESP8266 measures the time taken for the signal to return.

Step 3: Distance Calculation

The controller calculates the distance between the sensor and water surface.

For example:

Sensor reference distance = 100 cm

Measured water distance = 40 cm

Therefore:

Water Level = 100 - 40

= 60 cm

Step 4: Condition Detection

The ESP8266 compares 60 cm with the programmed thresholds.

Since 60 cm is between 50 cm and 80 cm, the system identifies the condition as:

WARNING

Step 5: Local Warning

The red LED is activated and the buzzer provides an intermittent warning.

The OLED displays:

WATER LEVEL SYSTEM

Level: 60 cm

Distance: 40 cm

Status: WARNING

Step 6: Critical Flood Detection

If the water rises to 80 cm or higher, the system changes to critical mode.

The OLED displays:

Level: 85 cm

Status: FLOOD ALERT

The red LED and buzzer are activated.

Step 7: SMS Notification

The ESP8266 sends AT commands to the GSM module.

The GSM module sends an SMS to the configured mobile number.

Example:

FLOOD ALERT!

Critical water level detected.

Current level: 85 cm.

Take immediate action.

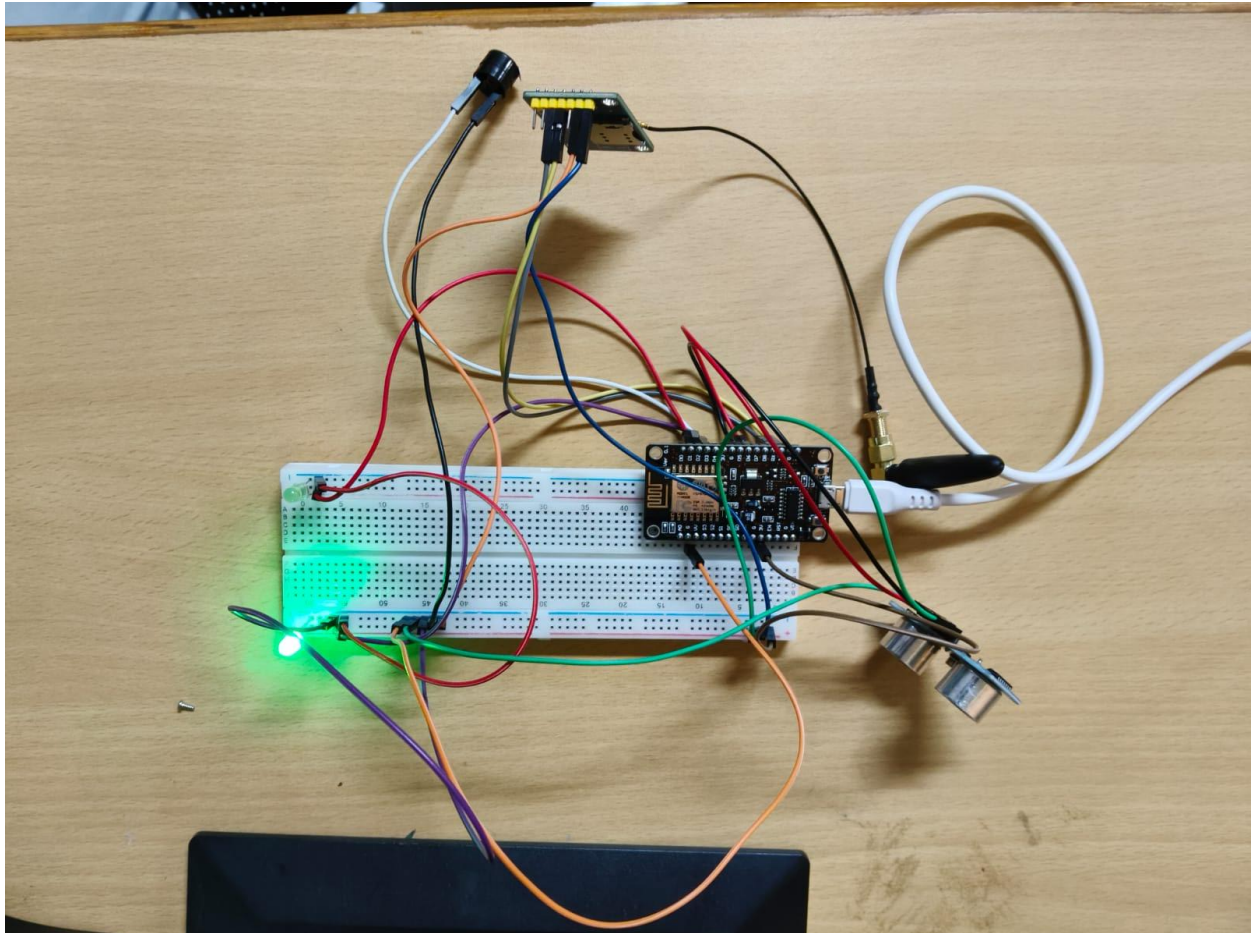
Step 8: Continuous Monitoring

After processing the current reading, the system returns to the measurement stage and continues monitoring.

Bill of Materials.:

Component	Quantity
ESP8266 NodeMCU	1
HC-SR04 Ultrasonic Sensor	1
Buzzer	1
LED Indicators	2
OLED Display	1

Prototype Implementation:



Results & Performance Analysis:

Detection

- The HC-SR04 provides non-contact measurement of the water surface.

Response Time

- The system can update the water-level status within a short period after each measurement.

Alert Mechanism

- Three forms of notification are available:
 - OLED display.
 - LED indication.
 - Buzzer alarm.
- The GSM module provides an additional remote SMS notification.

Automation

- The system automatically monitors the water level without requiring continuous human observation.

Remote Alert

- The GSM module allows the user to receive a warning even when they are away from the physical installation.